Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Danielle Hannold	dhannold@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I would like to create a game that tests how many basic mathematical operations a user can complete in 1 minute. They can continue to try to improve their scores, thereby getting quicker at computing.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

Math Mammoth online multiplication tables practice, IXL online practice, Math is Fun Math Trainer, XtraMath, and more.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

This application could be used in many different settings. Elementary/middle school teachers may require students to practice on the application to get better at their basic math facts. Teenagers/adults may compete to see who can do the most.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

My program will display a basic math expression, consume user answers, and return whether their answer is correct or incorrect, keeping track of how many answers are correct in one minute.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

They will need to decide which operation they want to work with (addition, multiplication, division, subtraction, or a mixture). They will then need to input their answers to each of the expressions presented to them.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The computer will have to randomly choose a mathematical expression for the user to solve, and decide whether their input matches the correct answer of the mathematical expression. It then needs to keep a running sum of their correct answers, and return this sum to the user. Finally, it will need to time the user for one minute.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get back their score for the round, and a question asking them if they would like to try again for an even better score.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things"

(nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

User name, Answer, expression/problem, user choices (operation), time in round, score

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Score is the most important. Details I need: Value of the score, is the score greater than a previous score, what does the score mean (how many were right/how many were attempted) Details I will ignore: how many times it took to get the score, timestamps of where each point was earned.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Randomly generating the mathematical problems seems the most difficult, and matching the user's answer to the correct answer of that randomly generated problem.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I plan to use an LLM to help me to code this project, however, in order to avoid "hallucinations", I plan to have an idea of how each part of the code should work before even using the LLM. If the code from the LLM matches what I expected, I will be more inclined to believe that it is correct, and I'll also know that I understand the code.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

My goal is to create something that I have an interest in, and maybe something I can play with my friends or future students, as I am a math education major.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

“I want to build a web application that generates random, basic math problems for users to solve, and keeps track of how many they get right in one minute. The math problems will be grouped depending on what the user chooses (multiplication, division, addition, subtraction, or combination. What are three features my application should have?”

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

Here are three features that would make your app more useful and engaging:

1. Difficulty levels (or number range selection)
Let users choose how hard the problems are, such as single-digit, double-digit, or custom ranges (e.g., times tables up to 12). This keeps the app useful for both a second grader practicing facts and an adult sharpening mental math. For division, it’s worth generating problems that always have whole-number answers so users aren’t stuck with remainders.

2. Score history and personal bests
After each one-minute round, show the results (correct, incorrect, accuracy) and save them so users can see progress over time. Tracking a personal best for each operation type gives people a concrete goal to beat and a reason to come back. Even simple storage, like a list of past scores per category, goes a long way.

3. Instant feedback and a clear timer with a results review

Show a visible countdown, immediately indicate whether each answer was right or wrong (a quick color flash works well), and let the next problem appear automatically. At the end, display a summary that lists the problems the user missed along with the correct answers, so they can learn from mistakes rather than just see a number.

Bonus ideas if you want to go further: a streak counter, sound effects with a mute option, keyboard-only input (Enter to submit) for speed, and a leaderboard if you add user accounts.

If you'd like, I can help you sketch out the app's structure or build a working prototypes

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

"The difficulty levels may be too complicated for an introductory programming project. This is because I would then have to limit the randomness of the program, which is something I do not necessarily know how to do, and I would have to determine different difficulty levels for each operation, which may be a lot. Could you give me a simpler alternative?"

AI response:

That's a reasonable call. Here's a simpler replacement for the difficulty feature.

Replacement: a streak counter

Show how many answers in a row the user has gotten right. It's one variable:

- *On a correct answer, add 1 to* `streak`*.*
- *On a wrong answer, set* `streak` *back to 0.*
- *Keep a second variable,* `bestStreak`*, and update it whenever* `streak` *exceeds it.*

It adds a fun challenge without any extra logic for different operations, and it works the same way for every problem type.

How to skip difficulty levels entirely

Pick one fixed number range for everything, such as 1 to 12 (the classic times-table range). Then you only need one random-number function, and every operation uses it. For division, a common beginner trick is to generate the answer and the divisor randomly, then multiply them to get the dividend. That guarantees whole-number answers without any complicated logic. For example, pick 7 and 6, then ask "42 ÷ 6" with the answer being 7.

A small thing to know for later

Limiting randomness is easier than it sounds. Choosing a random number between a minimum and maximum is usually a single line in most languages. So if you ever want to add difficulty levels, you can add them without rewriting your app: just let the user choose the min and max values. For now, though, keeping a fixed range is a perfectly good way to start.

If you tell me which language you're using, I can show you what the random number and division logic would look like.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I now have a better idea of what different aspects of my application will look like. For example, I know how the score will be presented and stored for users, and I like the idea of a streak counter. Also, the AI helped me to understand how to limit randomness for certain values, which will be nice to note. Overall, I just have a better idea of how my application will "flow"

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will go to office hours or pose my problem in the Discord server. Since I am allowed to use an LLM, if I feel that I cannot get past a certain point, I can help it point me in the right direction.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

1. *Ask LLM to recheck its code before using it.*

If there is still an error:

2. *Trace through the entirety of the code, ensuring I understand what each line is supposed to do*
3. *Determine if each line is doing what it is meant to do.*
4. *If I cannot find the issue, go to office hours and see if an instructor/TA can help find the issue.*

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

My goal in creating my drafter application is: "I would like to create an online calculator that can solve anything from basic addition to integrals. I would like to incorporate a graphing portion of the calculator, making it easy to solve systems of equations. This aligns with my academic interests, as I am a math education major, and this could be something beneficial for my students in the future." What input would the user need to give for this application?

For your Drafter calculator application, the user would need to enter the mathematical expression or equation they want to solve, along with any additional information needed for the calculation or graph.

Here is a response you could use for Part 2, Question 2 of your project worksheet:

Input: The user will enter a mathematical expression or equation into a text box, such as `5 + 3`, `x^2 + 2x + 1`, or `2x + y = 6`. Depending on the type of problem, the user may

also need to select a mathematical operation, enter the variables and their values, or specify the interval for a graph or integral.

Examples of user inputs

- Basic calculations: Numbers and operations, such as `12 / 4` or `7 * 8`.
- Algebra: Equations or expressions, such as `2x + 5 = 15`.
- Graphing: An equation, such as `y = x^2`, and a range of x-values to display.
- Systems of equations: Two or more equations, such as `y = 2x + 1` and `y = -x + 4`.
- Integrals: An expression to integrate, with upper and lower bounds for a definite integral.

A practical suggestion for your project: Since this is a first programming project using Drafter, I recommend starting with basic arithmetic and graphing simple functions. Symbolic integration and solving arbitrary mathematical expressions are considerably more complex, so you could treat them as future features rather than requirements for the first version.

I changed my goal to this: "I would like to create a game that tests how many basic mathematical operations a user can complete in 1 minute. They can continue to try to improve their scores, thereby getting quicker at computing. " At a high level, how will the computer manipulate or evaluate user input, if I said they are inputting "which operation they want to work with (addition, multiplication, division, subtraction, or a mixture). They will then

need to input their answers to each of the expressions presented to them."

Processing: The computer will randomly generate mathematical expressions based on the operation selected by the user. It will compare each answer the user enters with the correct answer, keep track of how many questions they answer correctly, and run a timer for one minute. When the time runs out, the program will display the user's final score so they can try again and improve their performance.

What then are some nouns that my program needs to track?

Some important nouns (things your program needs to track) for your math game are:

1. User — the person playing the game.
2. Math Problem — the randomly generated expression the user must solve.
3. Operation — the type of math problem (addition, subtraction, multiplication, division, or mixed).
4. Answer — the user's submitted answer and the correct answer for comparison.
5. Score — the number of questions answered correctly.
6. Timer — the countdown that tracks the one-minute game.
7. Game Session — one attempt at the game, including its score and selected operation.

For your project worksheet, I would focus on Math Problem, Answer, Score, Timer, and Game Session as the core things your program needs to track.

You don't necessarily need to create a separate Python class or complex data structure for each noun. In Drafter, these can be represented with simple variables and state values. For example, your score could be an integer, your timer could store the remaining seconds, and your selected operation could be a string.

Convo in part 5 with Claude:

Claude

Welcome, Danielle! I'm Claude.

Bring me anything—a tough problem, a half-formed idea, something you need to write. We'll figure it out together.

Where do you want to start?

I want to build a web application that generates random, basic math problems for users to solve, and keeps track of how many they get right in one minute. The math problems will be grouped depending on what the user chooses (multiplication, division, addition, subtraction, or combination. What are three features my application should have?

Here are three features that would make your app more useful and engaging:

1. Difficulty levels (or number range selection)
Let users choose how hard the problems are, such as single-digit, double-digit, or custom ranges (e.g., times tables up to 12). This keeps the app useful for both a second grader practicing facts and an adult sharpening mental math. For division, it's worth generating problems that always have whole-number answers so users aren't stuck with remainders.

2. Score history and personal bests
After each one-minute round, show the results (correct, incorrect, accuracy) and save them so users can see progress over time. Tracking a personal best for each operation type gives people a concrete goal to beat and a reason to come back. Even simple storage, like a list of past scores per category, goes a long way.

3. Instant feedback and a clear timer with a results review
Show a visible countdown, immediately indicate whether each answer was right or wrong (a quick color flash works well), and let the next problem appear automatically. At the end, display a summary that lists the problems the user missed along with the correct answers, so they can learn from mistakes rather than just see a number.

Bonus ideas if you want to go further: a streak counter, sound effects with a mute option, keyboard-only input (Enter to submit) for speed, and a leaderboard if you add user accounts.

If you'd like, I can help you sketch out the app's structure or build a working prototype.

"The difficulty levels may be too complicated for an introductory programming project. This is because I would then have to limit the randomness of the program, which is something I do not necessarily know how to do, and I would have to determine different difficulty levels for each operation, which may be a lot. Could you give me a simpler alternative?"

Simplifying difficulty levels into one easy feature.

Simplifying difficulty levels into one easy feature.

That's a reasonable call. Here's a simpler replacement for the difficulty feature.

Replacement: a streak counter

Show how many answers in a row the user has gotten right. It's one variable:

- On a correct answer, add 1 to `streak`.
- On a wrong answer, set `streak` back to 0.
- Keep a second variable, `bestStreak`, and update it whenever `streak` exceeds it.

It adds a fun challenge without any extra logic for different operations, and it works the same way for every problem type.

How to skip difficulty levels entirely

Pick one fixed number range for everything, such as 1 to 12 (the classic times-table range). Then you only need one random-number function, and every operation uses it. For division, a common beginner trick is to generate the answer and the divisor randomly, then multiply them to get the dividend. That guarantees whole-number answers without any complicated logic. For example, pick 7 and 6, then ask "42 ÷ 6" with the answer being 7.

A small thing to know for later

Limiting randomness is easier than it sounds. Choosing a random number between a minimum and maximum is usually a single line in most languages. So if you ever want to add difficulty levels, you can add them without rewriting your app: just let the user choose the min and max values. For now, though, keeping a fixed range is a perfectly good way to start.

If you tell me which language you're using, I can show you what the random number and division logic would look like.

Claude is AI and can make mistakes.

Claude finished the response

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.